

L1-zkEVM breakout #08

August 2026



Agenda

- Intro
- Projects updates
- *Observations from Observing zk-Proving* (Chetany)
- Q&As

EF blog post regarding Hegota



Categories ▾ Languages 🌐

Q search 🔍

⊛

EF Protocol: The Hegotá EIP Opinion Post and Tier List

The EF Protocol cluster's tier list for Hegotá evaluates 62 proposed EIPs in a unified view.

Posted by Ethereum Foundation Protocol Cluster on September 7, 2026

Research & Development



Table of contents ▾

This is the EF Protocol cluster's tier list for Hegotá. We evaluated the 62 EIPs proposed for inclusion, each with a tier and a short note on the grade.

It is the first time the cluster has published one unified view rather than per-team opinions. Geth, of course, being an EL client will still ship their own standalone tier list. However, they and the rest of the Protocol cluster (about 60 people in total) across research and engineering worked together to include their feedback in this post as a datapoint along with that of every other team and individual contributor that decided to participate.

8025	Optional Execution Proofs	A	2.38 (8)	DA & proofs	Upstreams stateless-execution changes into the canonical execution specs so zkVM work stops living on divergent branches.
8200	EVMification	C	1.43 (7)	Precompiled & cryptography	Every replacement bytecode must be audited to match precompile behavior exactly, and the bytecodes are not yet in the EIP. There is no hybrid: all clients run the bytecode or gas cannot be computed consistently. Graded with EIP-7666.
7709	Read BLOCKHASH from Storage and Update Cost	B	1.67 (6)	Repricing	Real value for the history-expiry direction; grades span three tiers. This EIP could move up to A-tier if devnet sequencing moves smoothly and allows for it to be added without delaying schedules.

<https://blog.ethereum.org/2026/09/07/protocol-hegota-eips>

EIP-8025 GH repo released!

<https://github.com/eth-act/eip-8025>

ACD.md

Case for CFI of EIP-8025

We believe the work completed so far justifies proposing EIP-8025 for CFI in Hegotá.

EIP-7723 requirement	Where EIP-8025 stands
Proposed for Inclusion, and reviewed by client developers	Presented at ACDC #178 on May 14, 2026
A Python implementation with tests in <code>execution-specs</code> , submitted as an open PR (<code>SHOULD</code>)	execution-specs#2288 is open, carrying the guest program, stateless interfaces, host-side witness construction, and ~163 conformance tests, with <code>tests-zkevm</code> releases maintained since April 2025. It is a draft, based on <code>forks/amsterdam</code> .
Intent to attempt inclusion in devnets	Already rehearsed rather than intended: Kurtosis <code>ethereum-package</code> support, stateless-input artifacts for <code>glamsterdam-devnet-5</code> and <code>-7</code> , and Lighthouse and Prysm interoperating with a GPU prover. More details in the readiness section .
Updates at this stage accompanied by updates to implementation and tests (<code>SHOULD</code>)	Already the working for at least six months of rebases across Glamsterdam devnets, owned by the zkEVM team

The consensus-layer half went further than CFI asks: [specs/features/eip8025](#) is merged in `consensus-specs` master. The work that remains is concentrated in `execution-specs`, mainly with the intention of upstreaming code.

ACD Q&A

This section addresses questions we have received about EIP-8025 in Hegotá, along with others we expect ACD participants to raise.

Why does EIP-8025 need to be scheduled in a fork?

EIP-8025 does not strictly *need* a hard fork from the perspective of protocol changes. What it needs is a formal review, upstreaming, and integration capacity from specification, CL, and EL teams, along with support in ethPandaOps tooling.

Much of this upstream work is orthogonal to other EIPs in Hegotá. STEEL and ethPandaOps may be able to continue it in the background, but requesting capacity through ACD is more transparent and predictable. It is also fairer to work already scheduled for the fork, because EIP-8025 would no longer draw on unallocated resources.

We encourage core developers to ask the STEEL and ethPandaOps teams for their assessment. We have worked closely with both teams over the last six months, keeping them up to date while maintaining the forks and rebasing across Glamsterdam without requesting extra resources. With appropriate capacity from those teams, we believe the work is ready for a smooth upstreaming process.

Why consider an EIP that only is essentially a request for resources?

The use of execution proofs to scale Ethereum has been under consideration [for more than eight years](#). It seems reasonable to consider derisking steps, like EIP-8025, on a case-by-case basis so that we can properly gather data and maintain extremely high

PROGRESS.md

- [eth-act/zkevm-standards](#).
- Testing:** The [RISC-V Compliance Test Monitor](#) publishes ISA compliance results for supported zkVMs ([source repository](#)).
- Formal verification progress (Planned!).

Infrastructure and tooling

- Proving:**
 - [eth-act/ere](#) — Provides a unified interface and toolkit for compiling, executing, proving, and verifying programs across multiple zkVMs.
 - [eth-act/ere-guests](#) — Maintains reusable libraries and compilation tooling for Ethereum guest programs across zkVMs with Ere.
 - [eth-act/zkevm-benchmark-workload](#) — Benchmarks Ethereum guest programs execution, proving, and verification across multiple zkVMs using EEST test and benchmark releases, and real mainnet/devnet blocks.
 - [eth-act/zkboost](#) — Implements an EIP-8025 proof node that serves consensus-client requests for execution-proof generation and verification through Ere-backed zkVMs.
 - [ethpandaops/proffessor](#) — Requests proofs of Ethereum execution blocks from zkboost and tracks each request through completion.
- Devnets:**
 - Mocked and real proving support in the Kurtosis [ethereum-package](#).
 - Stateless-input artifacts allowing EIP-8025 proving on Glamsterdam devnets:
 - [R2 bucket for glamsterdam-devnet-5 stateless inputs](#)
 - [R2 bucket for glamsterdam-devnet-7 stateless inputs](#)

Coordination: zkEVM breakout calls

Call	Date	Resources
7	August 12, 2026	Recording & notes · Slides
6	July 8, 2026	Recording & notes · Slides
5	June 10, 2026	Recording & notes · Slides
4	May 13, 2026	Recording & notes · Slides
3	April 8, 2026	Recording & notes · Slides
2	March 11, 2026	Recording & notes · Slides
1	February 11, 2026	Recording & notes · Slides

Further reading

For technical deep dives on EIP-8025, zkVM performance, interoperability standards, and security, see the [zkEVM Team blog](#).

zkEVM project 1, 2, 3 & 6

Execution Witness & Guest Program specs/test,
benchmarks/metrics

Specs & tests

- test-zkevm@v0.8.{[0](#), [2](#), [4](#)}
 - Based on *glamsterdam-devnet-8*
 - Make SCHEMA_ID part of public input.
 - Remove *ChainConfig* and only leave *chain_id*.
 - External t8n filling bug fixes
 - Fixes EIP-8282 new execution request not being considered
 - [glamsterdam-devnet-8 R2 bucket](#)
- [tests-zkevm-benchmark@v0.8.3](#)
 - 10M, 30M and 60M gas limits

R2 bucket: glamsterdam-devnet-8 stateless input

glamsterdam-devnet-8 stateless inputs

Batch-first public dataset catalog generated at 2026-09-07T19:50:30.140975408Z.

Block artifacts

20660

Batches

2057

Batch size

10

Total batch size

9.88 GiB

How to download

Each archive contains benchmark-ready EEST fixtures under `blockchain_tests/` and metadata at `.meta/manifest.json`.

```
curl -L0 exports/batches/93300-93309.tar.zst
tar --zstd -xf 93300-93309.tar.zst
```

Pass the extracted directory directly to `ere-hosts --input` folder.

Verify downloads with [SHA256SUMS](#).

Machine-readable indexes

- [manifest.json](#) Dataset manifest
- [batches.jsonl](#) Batch index

R2 public buckets do not provide directory listing; use these files instead of folder URLs.

Batch archives

Blocks	Artifacts	Size	SHA-256	Download
159380-159389	10	5.95 MiB	0x24c517567dd1c04f...	159380-159389.tar.zst
159370-159379	10	5.76 MiB	0x03e2d2a577a01008...	159370-159379.tar.zst
159360-159369	10	5.75 MiB	0x0c9b6b87466579c4...	159360-159369.tar.zst
159350-159359	10	5.51 MiB	0x9ece135ac2740f25...	159350-159359.tar.zst
159340-159349	10	5.96 MiB	0x1c42dea3a2cc2779...	159340-159349.tar.zst
159330-159339	10	5.84 MiB	0x8104f87565e0aec7...	159330-159339.tar.zst
159320-159329	10	5.71 MiB	0x6dc0bd7367101c48...	159320-159329.tar.zst

<https://pub-760ad8b3dd9547539f829c1ea30f18b5.r2.dev/devnets/glamsterdam-devnet-8/index.html>

R2 bucket: glamsterdam-devnet-8 stateless input

glamsterdam-devnet-8 stateless inputs

Batch-first public dataset catalog generated at 2026-09-07T19:50:30.140975408Z.

Block artifacts

20660

Batches

2057

Batch size

10

Total batch size

9.88 GiB

How to download

Each archive contains benchmark-ready EEST fixtures under `blockchain_tests/` and metadata at `.meta/manifest.json`.

```
curl -L0 exports/batches/93300-93309.tar.zst
tar --zstd -xf 93300-93309.tar.zst
```

Pass the extracted directory directly to `ere-hosts --input` folder.

Verify downloads with [SHA256SUMS](#).

Machine-readable indexes

- [manifest.json](#) Dataset manifest
- [batches.jsonl](#) Batch index

R2 public buckets do not provide directory listing; use these files instead of folder URLs.

Batch archives

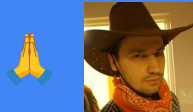
Blocks	Artifacts	Size	SHA-256	Download
159380-159389	10	5.95 MiB	0x24c517567dd1c04f...	159380-159389.tar.zst
159370-159379	10	5.76 MiB	0x03e2d2a577a01008...	159370-159379.tar.zst
159360-159369	10	5.75 MiB	0x0c9b6b87466579c4...	159360-159369.tar.zst
159350-159359	10	5.51 MiB	0x9ece135ac2740f25...	159350-159359.tar.zst
159340-159349	10	5.96 MiB	0x1c42dea3a2cc2779...	159340-159349.tar.zst
159330-159339	10	5.84 MiB	0x8104f87565e0aec7...	159330-159339.tar.zst
159320-159329	10	5.71 MiB	0x6dc0bd7367101c48...	159320-159329.tar.zst



Might be reset soon

<https://pub-760ad8b3dd9547539f829c1ea30f18b5.r2.dev/devnets/glamsterdam-devnet-8/index.html>

Potential DoS regarding guest program randomness availability



Guest data structure key seeding standards #41

Open

frisitano opened 5d ago

In zkEVM's guest clients, state entries are stored in bucketed data structures. @LukaszRozmej articulates the vulnerability and concern in this PR - [ethereum/EIPs#12286](#).

We should define a standard that provides pseudo-random seeding to prevent such a vulnerability.

Create sub-issue

frisitano mentioned this 5d ago

Update EIP-8025: require seeded container hashing in the guest ethereum/EIPs#12289

jsign 5d ago

Leaving my thoughts after a quick chat with @frisitano.

Feels like using `new_payload_request_root` should be enough of a pseudorandom seed to be used for guest program containers. Still, doesn't have a "formal" proof that can't be biased via a FOCIL tx in such way that is dangerous for a bunch of guest programs (or maybe even a single one); since the builder will still put other stuff in the execution payload. (A malicious builder prob is out of scope under the assumption it has to prove its own built blocks).

The "purest" solution is to maybe add a `random()` precompile that guest programs can use and zkVM provide, but that would add some complexity in the standards.

So maybe we need to come up with a good argument on why `new_payload_request_root` as seed isn't enough.

I guess we have three options:

1. Receive seed in StatelessInput (original Lukasz take)
2. This `new_payload_request_root` seed (if found secure)
3. `random()` precompile in standard (doesn't change the spec, and ask more from zkVMs)

benaadams 5d ago

I think prover-private cryptographically secure randomness is materially stronger than deriving the seed from `new_payload_request_root`.

With `new_payload_request_root`, every prover gets the same seed for the same payload. If that seed causes pathological collisions, all provers suffer the same slowdown and retries reproduce it.

With a fresh CSPRNG-derived private seed per proof attempt, different provers get independent hash layouts. If one gets unlucky, another prover - or a retry with a new seed - can complete normally.

The guest and program commitment remain unchanged, provided the seed only affects internal data-structure layout and not public output

<https://github.com/eth-act/zkevm-standards/issues/41>

feat: Host randomness standard #42

Draft codygunton wants to merge 2 commits into eth-act:main from codygunton:host-randomness

Conversation 0 Commits 2 Checks 10 Files changed 1

codygunton commented 3 days ago

This standard requires a zkVM to provide guests access to a cryptographically strong pseudorandom number generator (CSPRNG) via a function `zkvm_random_u64`. Cf [ethereum/EIPs#12289](#) by @LukaszRozmej

codygunton added 2 commits 3 days ago

add draft host randomness standard Verified ✓ 4e6c048

clarification Verified ✓ 16a6823

jsign mentioned this pull request 7 hours ago

Guest data structure key seeding standards #41

Open

<https://github.com/eth-act/zkevm-standards/pull/42>

(Some) Tooling repos

- [zkevm-benchmark-workload](#)
 - [Pending PR reg v0.8.x support](#) waiting for guest program readiness + devnet-8 “fork”.
- [ere-quests](#):
 - [All ELF's are pulled from guest program repo official releases!](#)